Native hashed-ecdl-tensor measurements on small prime-order curves

Study: `research/btn_ecdl_prime_20261029/` (PR #2888). Tool: `native/` (`btn-prime-scaling`, Rust), depending on the unmodified `crates/hashed-ecdl-tensor` crate (#2878). All numbers below are read from `native_runs/report.json`, produced by `btn-prime-scaling report native_runs` from the raw records `sizes.jsonl`, `counts.jsonl`, `solves.jsonl` and `eccp131_size.json`.

Unit. Step counts are the crate's `Meter` charges, deterministic diagnostics and not calibrated time units. **No wall-clock time is reported**, so no timing isolation applies. Host: a 4-core Claude Code cloud container (Linux).

Exact counts against brute force

16 contractions of the decomposition network (two slots, full and hash-filtered factor bases) at p = 5, 7, 11, 13. All match brute force: **`true`**. Backend `Functional` in every count: **`true`**. Visited assignments equal 2^{4d} in every count: **`true`**. Recursive (OpenAI Appendix D) backend used anywhere: **`false`**.

p	d	brute	backend	assignments	steps	steps / assignment
5	3	match	Some(Functional)	4096	4485600	1095
5	3	match	Some(Functional)	4096	4485600	1095
5	3	match	Some(Functional)	4096	3523331	860
5	3	match	Some(Functional)	4096	3544187	865
7	3	match	Some(Functional)	4096	9627056	2350
7	3	match	Some(Functional)	4096	9627056	2350
7	3	match	Some(Functional)	4096	6463893	1578
7	3	match	Some(Functional)	4096	6463893	1578
11	4	match	Some(Functional)	65536	69188532	1056
11	4	match	Some(Functional)	65536	69188532	1056
11	4	match	Some(Functional)	65536	58303887	890
11	4	match	Some(Functional)	65536	58318774	890
13	4	match	Some(Functional)	65536	107725686	1644
13	4	match	Some(Functional)	65536	107725686	1644
13	4	match	Some(Functional)	65536	93262386	1423
13	4	match	Some(Functional)	65536	93277273	1423

End-to-end solves (`reduction::solve`, seeded)

p	d	order r	seed	secret	recovered	relations	steps
5	3	7	1	2	2 (verified)	13	210937575
5	3	7	2	1	1 (verified)	13	211864590
7	3	13	1	3	3 (verified)	25	738643764

Network size

d	p	edges	vertices	max degree
3	5	31624	8402	132
4	11	56480	12860	132
5	17	57952	13722	132
6	37	59580	14662	132
7	67	61120	15558	132
8	131	64848	16552	132
9	257	114192	19438	132
10	521	115816	20376	132

11	1031	117440	21314	132
12	2053	118916	22178	132
13	4099	120456	23074	132
14	8209	122008	23976	132
15	16411	123648	25182	260
16	32771	127920	31176	260
17	65537	223736	35474	260

ECCp-131 (d = 131, published challenge input, certified primes): 2 slots, 1684796 edges, 249620 vertices, max degree 1032, 526 interface bits (4 · 131 free coordinate bits plus 2 constant identity flags).

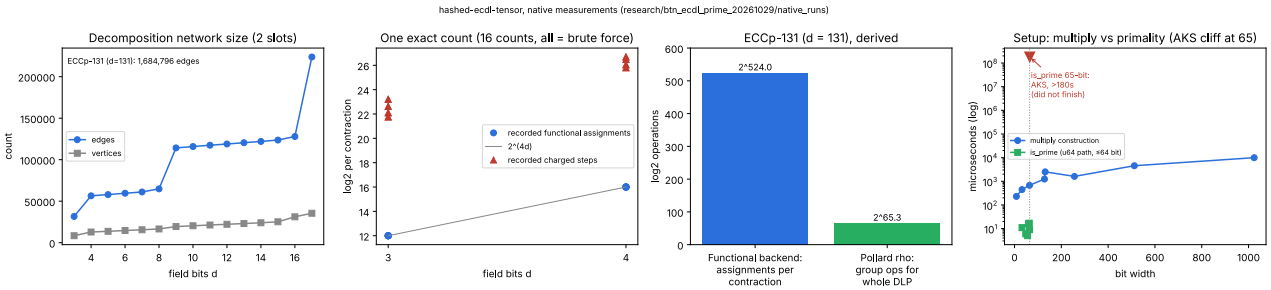


Figure 1: Recorded sizes, per-count assignments and steps, and the derived ECCp-131 comparison. Source: `native_runs/report.json`.

Setup: primality certification above 64 bits (resolved with certificates)

`is_prime` (used by `Curve::new` and `FactorBase::new`) dispatches to a fast u64 path at <= 64 bits and to **AKS** above.

Without certificates, every certification at <= 64 bits took at most 17 µs, while AKS at 65 bits did **not finish in 180 s**. That is why the first uncertified ECCp-131 build never finished.

The crate on `main` now accepts independently checked Pocklington certificates (`Curve::with_cofactor_and_certificates`, landed by a separate change). With the bundled `eccp131-certified-hash.json` (5 certificates, certified 132-bit hash prime) the ECCp-131 curve validates in about 13 ms and its full decomposition network builds (below). Network construction itself is fast at every width: the Fourier integer-multiply builds in 8-bit: 229 µs, 32-bit: 447 µs, 64-bit: 674 µs, 128-bit: 1244 µs, 131-bit: 2508 µs, 256-bit: 1610 µs, 512-bit: 4550 µs, 1024-bit: 9950 µs. So setup is no longer the wall; the recursion crossover is.

Speeding up the exact fallback that actually runs

At every reachable size the count runs on the exact `Functional` fallback, so that is where speed-ups land. Two were measured on the same 16 instances, in the crate's charged steps (deterministic `Meter` diagnostics, not time).

Upstream functional pruning (crate change landed separately, measured here): 16 of 16 instances paired, counts unchanged, charged steps 1.5–6.4× lower.

Derived last point (this study): for two slots the decomposition network searches both A_1 and A_2 as free inputs, but A_2 = target − A_1 is determined. Computing it with the encoder instead of searching it is a bijection on the same identity-free language, so counts cannot change, while the free input bits halve (d = 3: 12 ⟶ 6, d = 4: 16 ⟶ 8). Built only from the crate's public `Encoder` API; the crate is not modified.

- Counts equal to the original network and to brute force on all 16 instances: **`true`**.
- Lemma-1 witness extraction on the derived network: 15 decompositions found, every one checked by reference arithmetic and the crate's membership test (the remaining instance has none, as brute force agrees): **`true`**.
- Charged steps 9.1–65.2× lower than the original network, on top of the pruning; the gain grows with p.

p	threshold	count	free bits	assignments (orig ⟶ derived)	steps original	steps derived	ratio
5	11	5	12 ⟶ 6	295 ⟶ 43	2379059	163933	14.5×
5	11	5	12 ⟶ 6	295 ⟶ 43	2379059	163933	14.5×
5	5	0	12 ⟶ 6	181 ⟶ 41	1314140	135637	9.7×

5	5	2	12 → 6	181 → 41	1334996	146621	9.1×
7	17	11	12 → 6	729 → 57	6563062	262515	25×
7	17	11	12 → 6	729 → 57	6563062	262515	25×
7	8	2	12 → 6	324 → 48	2639459	203406	13×
7	8	2	12 → 6	324 → 48	2639459	203406	13×
11	23	9	16 → 8	1981 → 181	19748700	455268	43.4×
11	23	9	16 → 8	1981 → 181	19748700	455268	43.4×
11	11	2	16 → 8	984 → 164	9068321	389880	23.3×
11	11	3	16 → 8	984 → 164	9083208	397607	22.8×
13	29	17	16 → 8	3991 → 211	42651459	653941	65.2×
13	29	17	16 → 8	3991 → 211	42651459	653941	65.2×
13	14	6	16 → 8	2314 → 178	23810227	530356	44.9×
13	14	7	16 → 8	2314 → 178	23825114	538083	44.3×

At ECCp-131 the exact fallback's worst case drops from 2^{524} to 2^{262} assignments per count. That is a large constant-exponent improvement and still far beyond Pollard rho's $2^{65.3}$ group operations for the whole problem; it does not change the conclusion below.

The recursion crossover (headline)

The only part of the algorithm that could beat exponential cost is the recursive backend (OpenAI Appendix D.4). In the crate's `Counter::count`, it is selected only when root_u $= \lambda/131072$ is at least 1, with $\lambda = \lceil \log_2(2 \cdot \text{edges}) \rceil$. That needs $\lambda \geq 131072$, i.e. at least 2^{131070} edges (further gates only raise it).

The largest network built here is $2^{20.7}$ edges. The gap to the crossover is about 2^{131049}. Across all 16 counts and 3 solves the recursive backend was selected **`false`** times: every exact count used the fallback (`Functional`/`Direct`), whose cost is $2^{\text{free input bits}}$ per contraction.

This threshold is faithful to the manuscript, which states a saving only above networks of size 2^17 = 131072 (manuscript's 2^(2^17)). No prime-field curve produces a network near it, so this implementation gives no sub-exponential behaviour at any reachable size, including ECCp-131.

Derived: ECCp-131

The backend actually used at every measured size is `Functional`, which visits $2^{\text{free input bits}}$ assignments. With two slots at d = 131 that is 2^{524} assignments per contraction (Functional backend visits 2^(free input bits); measured equal to 2^(4d) at d=3,4; two slots at d=131 give 4*131 = 524 free bits). With A_2 derived (above) it is 2^{262}. Pollard rho on ECCp-131 needs about $2^{65.33}$ group operations for the whole discrete logarithm. The recursive backend did not run at any measured size; this report says nothing about its cost.

What this does not show

No ECCp-131 scalar is computed or attempted, and nothing here approaches Pollard rho. The two fallback speed-ups above are measured in the crate's charged steps on small curves, against the same code's earlier behaviour; they are not wall-clock benchmarks and not a comparison with rho. The construction and primality timings are unisolated diagnostics. The recursive backend's asymptotic bound (Bochum v0.2, conditional on OpenAI's unreviewed Theorem D.1) is not tested. No row is added to `benchmarks/README.md`: the validation gate admits no row without an independent replay certificate, validator report and matched rho reference.